

CS-20200 Term Project — Requirements Document

Project Title: Absurdle — an Adversarial Word Guessing Game

Overview

Absurdle is a single-player word guessing game played in the terminal. The player tries to guess a 5-letter secret word. After each guess, the game colors each letter to show how close the guess is: green for a correct letter in the correct position, yellow for a correct letter in the wrong position, and gray for a letter that does not appear in the secret word. So far this is just Wordle.

The twist is that the game does not actually pick a secret word at the start. Instead, it keeps a set of all possible secret words and, after each guess, picks whichever feedback pattern keeps that set as large as possible. The game is therefore adversarial: it delays committing to a real answer for as long as it can, while staying consistent with every piece of feedback it has already given. The player wins when only one word is left in the set *and* their last guess equals that word.

There is no guess limit. The challenge is to win in as few guesses as possible.

Requirements

- **R1.** The program reads its word list from a text file `words.txt` in the working directory. Each line contains one 5-letter lowercase English word. This file is shipped in the repository and contains the list of allowed guesses and possible answers.
- **R2.** When the program starts, it prints a short title banner and a one-line instruction explaining that the player must type a 5-letter word and press enter.
- **R3.** The program prompts the player with *Guess #N:* (where N is the current guess number, starting at 1) and reads one line from standard input. Input is case-insensitive and whitespace is trimmed before validation.
- **R4.** A guess is valid if it is exactly 5 letters long and appears in `words.txt`. If the input is not 5 letters, the program prints *Guess must be exactly 5 letters.* and re-prompts. If the input is 5 letters but not in the word list, the program prints *Not in word list.* and re-prompts. The guess number does not advance until a valid guess is entered.
- **R5.** Internally, the game maintains a *candidate set*: the set of words still consistent with all feedback given so far. At the start of the game, the candidate set is the entire word list.
- **R6.** When the player submits a valid guess, the game does the following. For every possible feedback pattern (each pattern is a string of 5 colors, each color in *green*, *yellow*, or *gray*), it computes how many words in the current candidate set would produce that pattern when compared against the guess. The game then picks the pattern with the

largest such count. Ties are broken by preferring the pattern with the fewest greens; if still tied, the fewest yellows; if still tied, lexicographic order of the pattern string with *gray* < *yellow* < *green*. The candidate set is then replaced by exactly the words that produce the chosen pattern.

- **R7.** After computing the chosen pattern, the program prints the guess with each letter colored according to that pattern. Green letters use a green background, yellow letters use a yellow background, and gray letters use a gray background, all rendered with ANSI escape codes. The colored guess is appended below previous guesses so the player sees the full history.
- **R8.** The player wins if and only if (a) the candidate set has shrunk to exactly one word *and* (b) the just-submitted guess equals that one word. When this happens, the program prints *Solved in N guesses!*, where N is the total number of guesses made, and ends the game.
- **R9.** If the candidate set shrinks to exactly one word but the player's guess is not that word, the game continues normally; the player must guess the remaining word explicitly to win.
- **R10.** After every guess that does not win the game, the program prints the size of the current candidate set on its own line as *Possible answers remaining: K*. This lets the player see how much progress each guess made.
- **R11.** At any prompt, the player may type `:quit` to exit the program immediately. The program prints *Goodbye.* and exits with status 0.
- **R12.** At any prompt, the player may type `:reveal` to give up. The program then prints one word still in the candidate set, prefixed with *The answer could have been:*, and ends the game.
- **R13.** After the game ends (win or reveal), the program asks *Play again? (y/n):* and starts a fresh game on input **y** or **Y**. Any other input exits the program.
- **R14.** The program supports an optional `--seed N` command-line argument. The seed only affects tie-breaking when reading `:reveal` output (which word from the candidate set is shown), so two runs with the same seed and same player inputs produce identical output.
- **R15.** The implementation is written in F# and targets .NET 10. The project builds and runs with only the .NET 10 SDK installed; no other system dependencies are needed.
- **R16.** The repository contains the F# source code, the `.fsproj` file, this requirements PDF, the `words.txt` word list, and a `README.md` with build and run instructions.

Example Interaction

The program prints a title banner and the instructions. It prompts *Guess #1:* and the player types **crane**. The game compares the guess against every word in the candidate set, picks the feedback pattern that keeps the candidate set as large as possible (in this example, all letters gray), prints **crane** with all letters on a gray background, and prints *Possible answers remaining: 5762*. The player tries **moist**; this time the chosen pattern marks the **o** green and

the rest gray, so the program prints **moist** colored accordingly and *Possible answers remaining: 31*. After several more guesses the candidate set shrinks to one word; the player guesses that word and the program prints *Solved in 7 guesses!* and asks whether to play again.

A Note on Scope

The adversarial logic in R6 is the project's central technical contribution. It is a small adversarial-search algorithm: at every step the 'host' chooses the feedback that maximally postpones commitment to a single answer, which is equivalent to the host playing optimally in a maximum-game-length sense. The implementation runs in time linear in the size of the candidate set per guess, which is fast enough for interactive play on any standard word list of a few thousand words. If, during implementation, the rendering of ANSI colors turns out not to work reliably in some terminals used by reviewers, the program may fall back to plain-text markers (e.g. G, Y, . below each letter) instead of background colors. Any such change will be recorded and explained in the README.md change log, as the course specification requires.